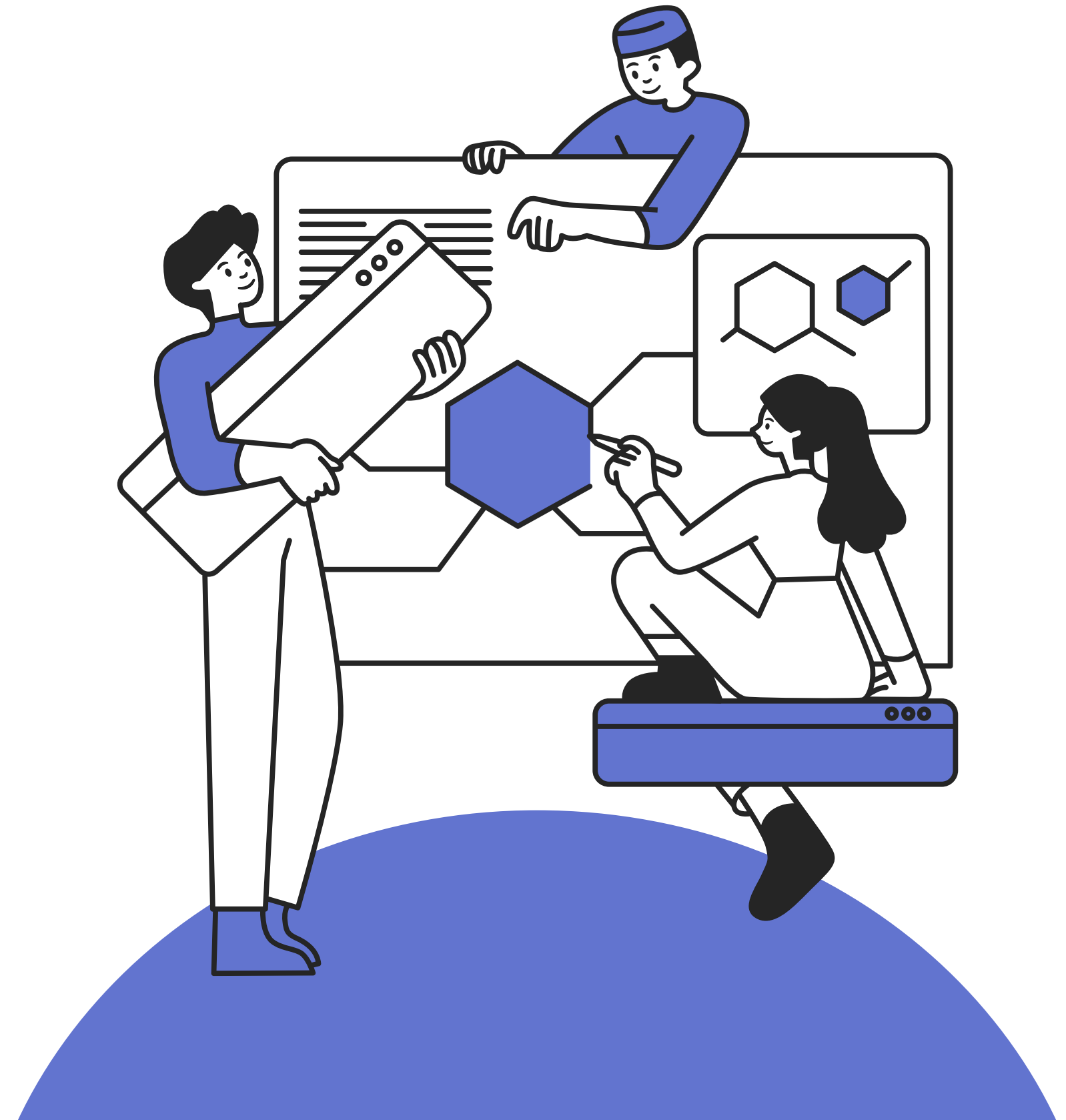
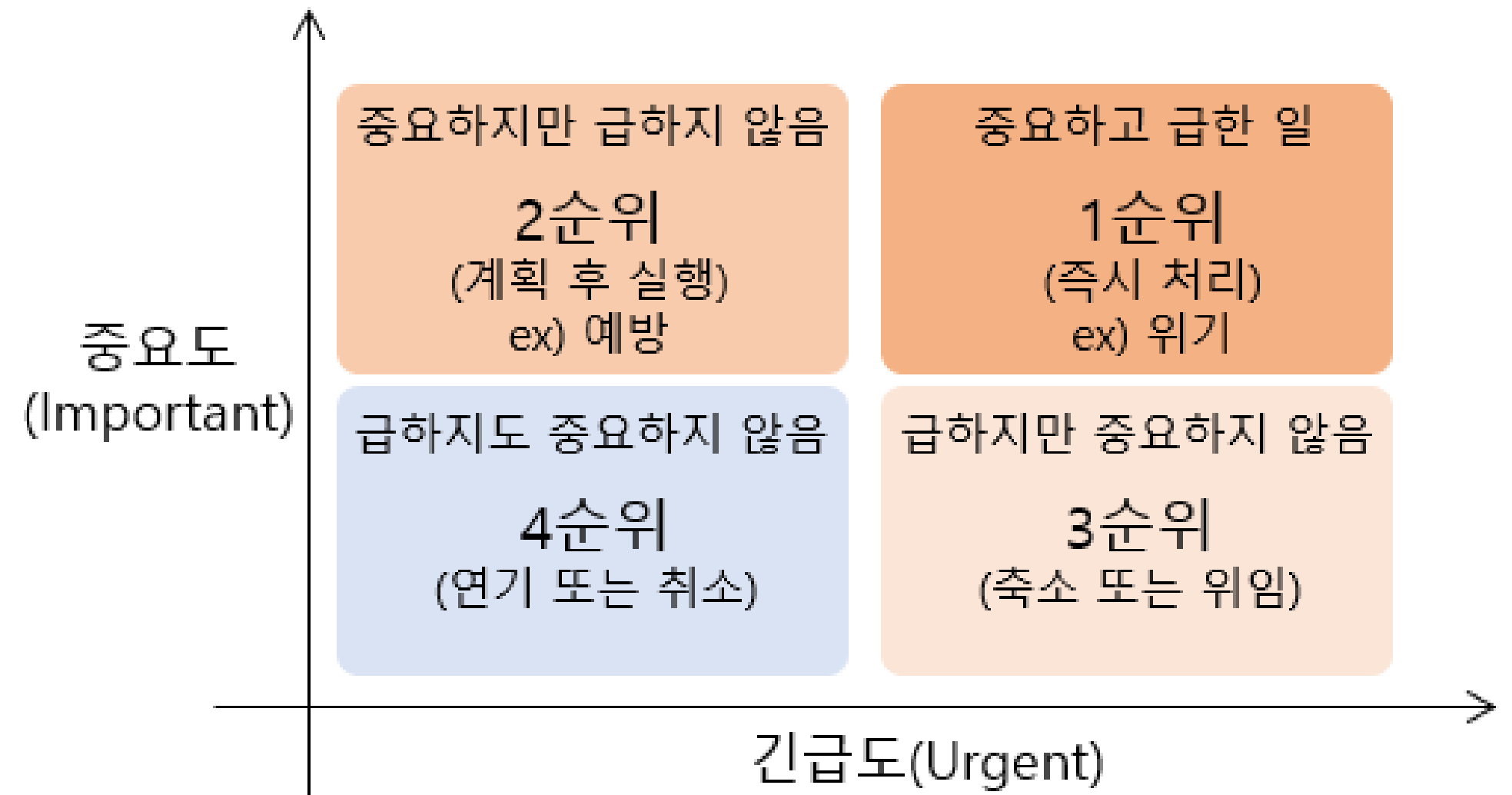
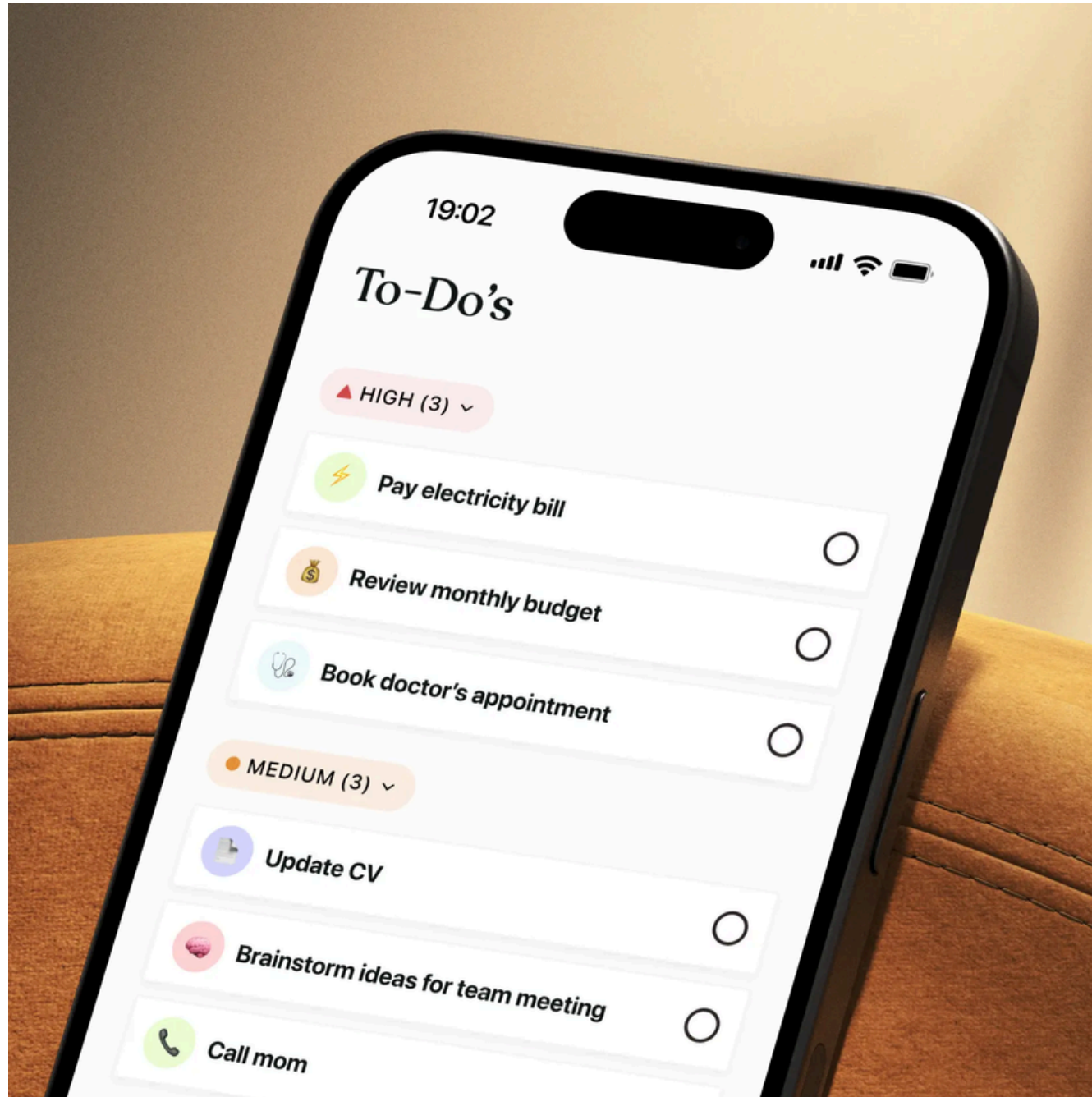


우선순위 큐와 힙

이예린T



어떤 일부터 해야 할까?



CONTENTS

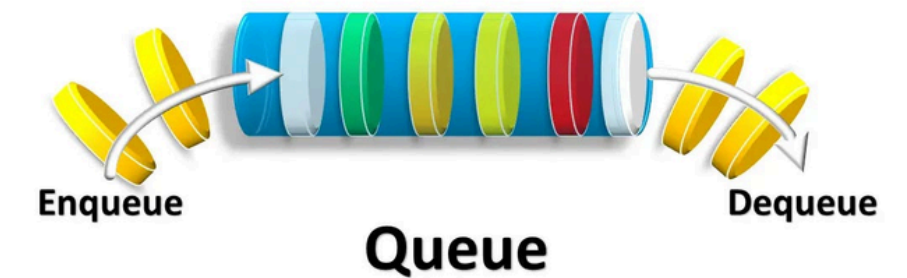
- 1 우선순위 큐의 소개
- 2 힙 트리를 통한 우선순위 큐 구현
- 3 모듈별 탐구
- 4 우선순위 큐와 힙 정리

01

우선순위 큐



큐는 FIFO



우선순위에 따라 처리한다.

- 우선순위 속성에 따라 출력 순서를 결정한다.
- 우선순위 속성을 갖는 데이터의 삽입과 제거 연산을 지원하는 ADT
- 새 요소에 우선순위를 부여해서 큐에 삽입하고 가장 높은 우선순위를 가진 요소부터 빠져나온게 한다.
- 언제든지 임의의 우선순위를 가진 원소를 우선순위 큐에 삽입할 수 있다.
- 우선순위란?
 - 여러 개의 작업이나 데이터가 있을 때 어떤 것을 먼저 처리할지 정하는 기준
 - 작은 수가 우선순위가 높은 경우: 프린터에서 작업 번호가 작을수록 먼저 출력 (Min priority)
 - 큰 수가 우선순위가 높은 경우: 게임에서 점수가 높은 사람이 먼저 보상 받게 함. (Max Priority)

CEO's top priorities for IT this year



Q: What are the CEO's top three priorities for you in the coming year?

CIO

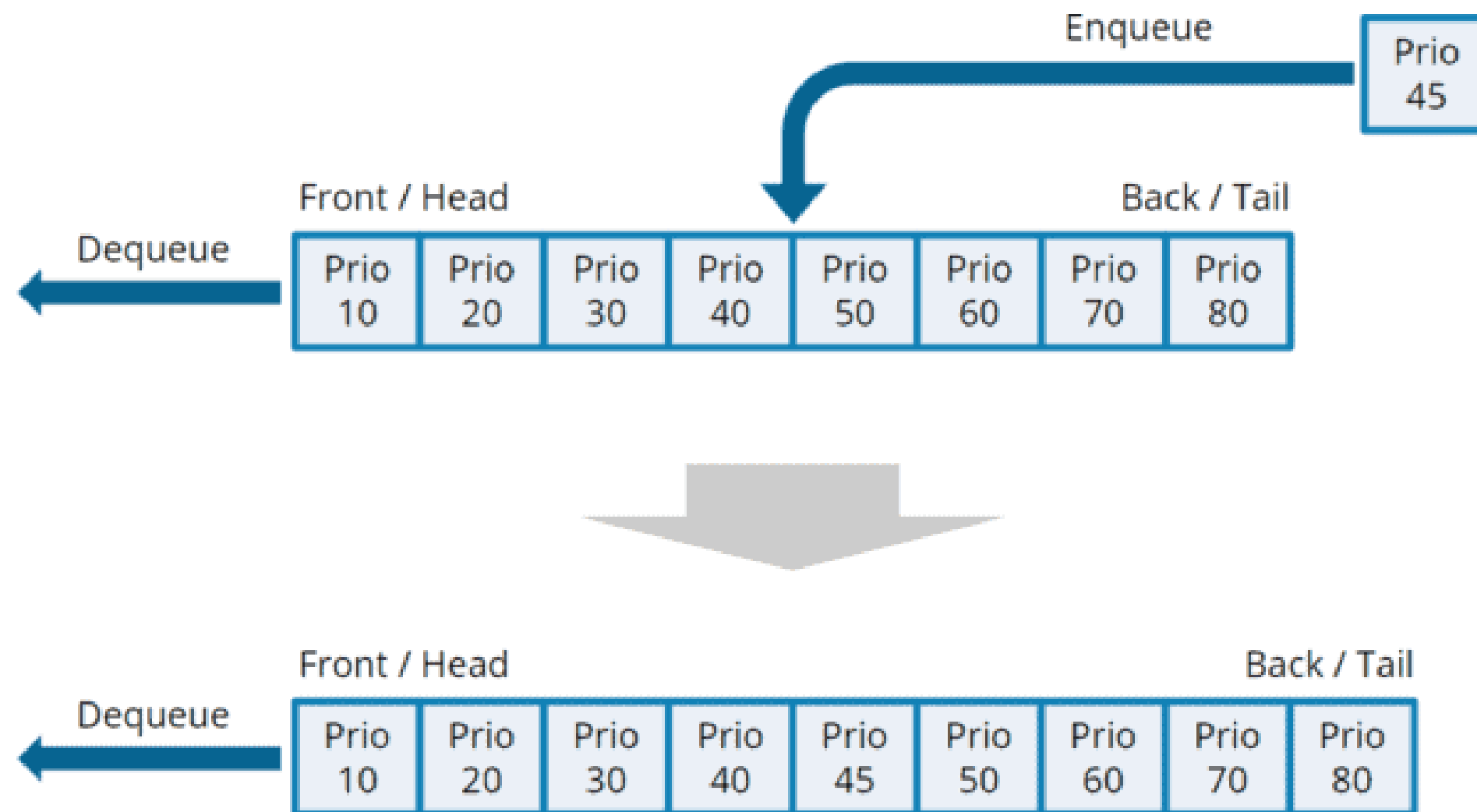
Source: State of the CIO, 2025

02

우선순위 큐의 ADT

코드) heapq 모듈 사용해 보기

- 삽입: 우선순위 큐에 값을 삽입한다.
- 제거: 우선순위 큐에서 우선순위가 가장 높은 값을 꺼낸다.
- 순회: 우선순위 큐의 값들을 순차적으로 출력한다.



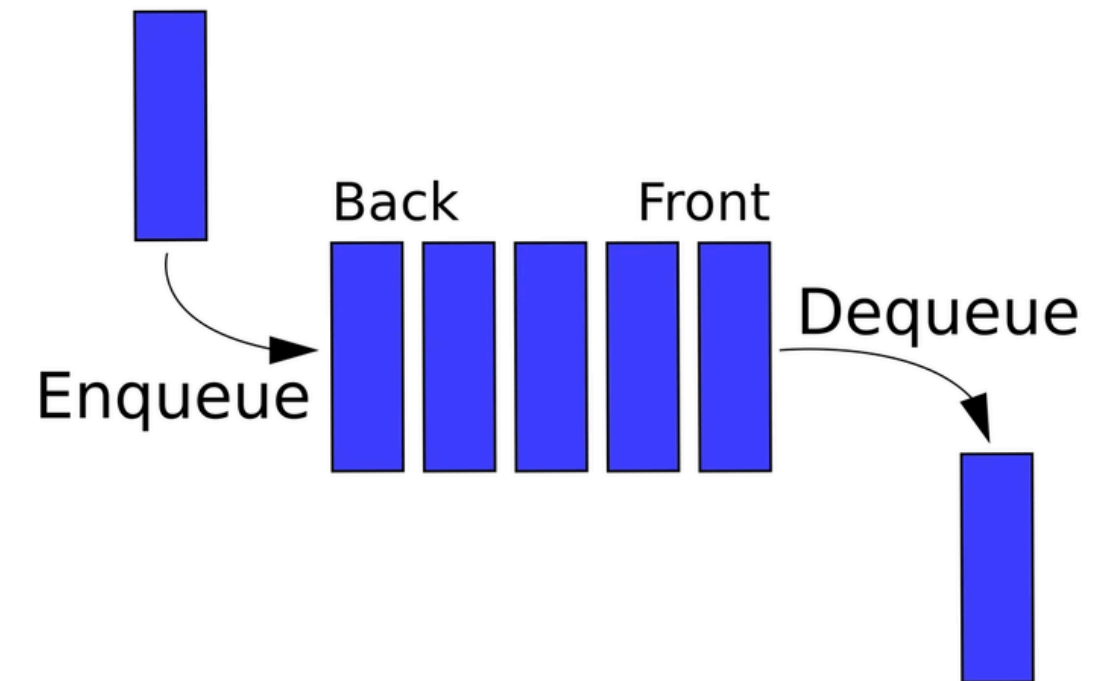
[부연 설명]

- 우선순위 큐 내부는 항상 우선순위 순서대로(head가 높은 우선순위) 정렬돼 있다.
- dequeue를 하면 가장 높은 우선순위의 값이 꺼내진다.
- enqueue를 할 때는 우선순위 정렬에 맞는 위치에 삽입된다.

03

우선순위 큐의 구현

- 지난 주에 구현했던 큐를 변형하여 우선순위 큐를 구현하시오.
- 필수 포함: enqueue, dequeue, 순회
 - 달라져야 하는 메서드는 하나 뿐
 - 숫자가 작을수록 높은 우선순위(Min priority)로 약속



03

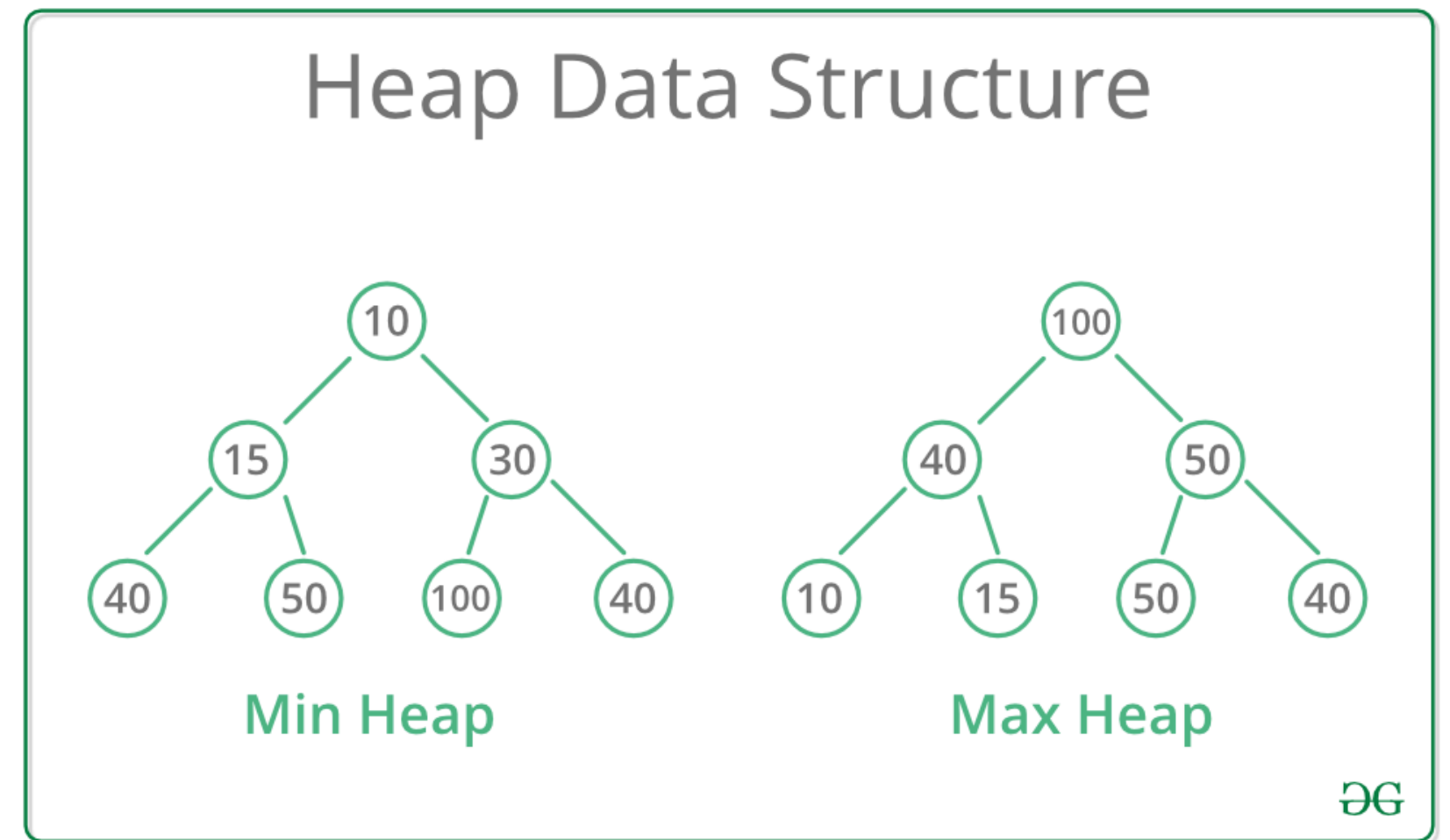
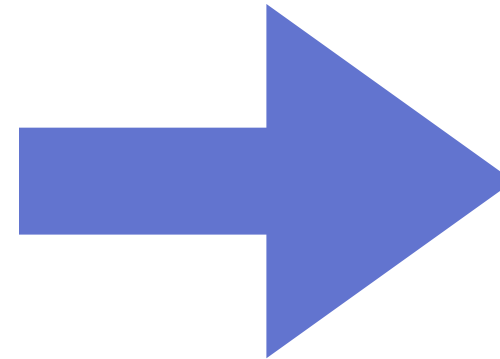
우선순위 큐의 구현

삽입, 삭제 속도를 줄이도록 새로운
자료구조 힙을 도입

문제가 있다!

이 자료구조는 과연 효율적인가?

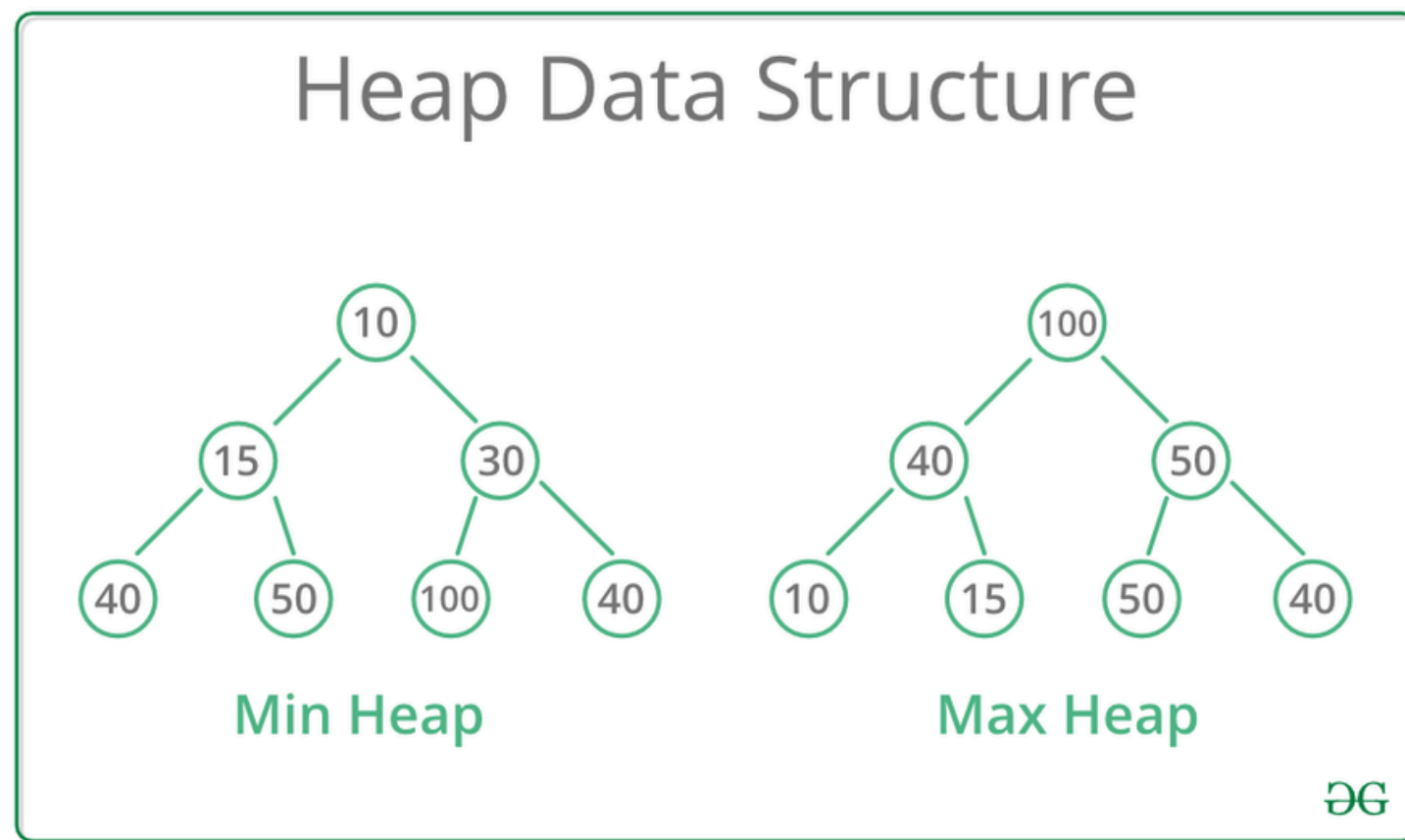
enqueue 시 최악의 경우?



01

힙(Heap)

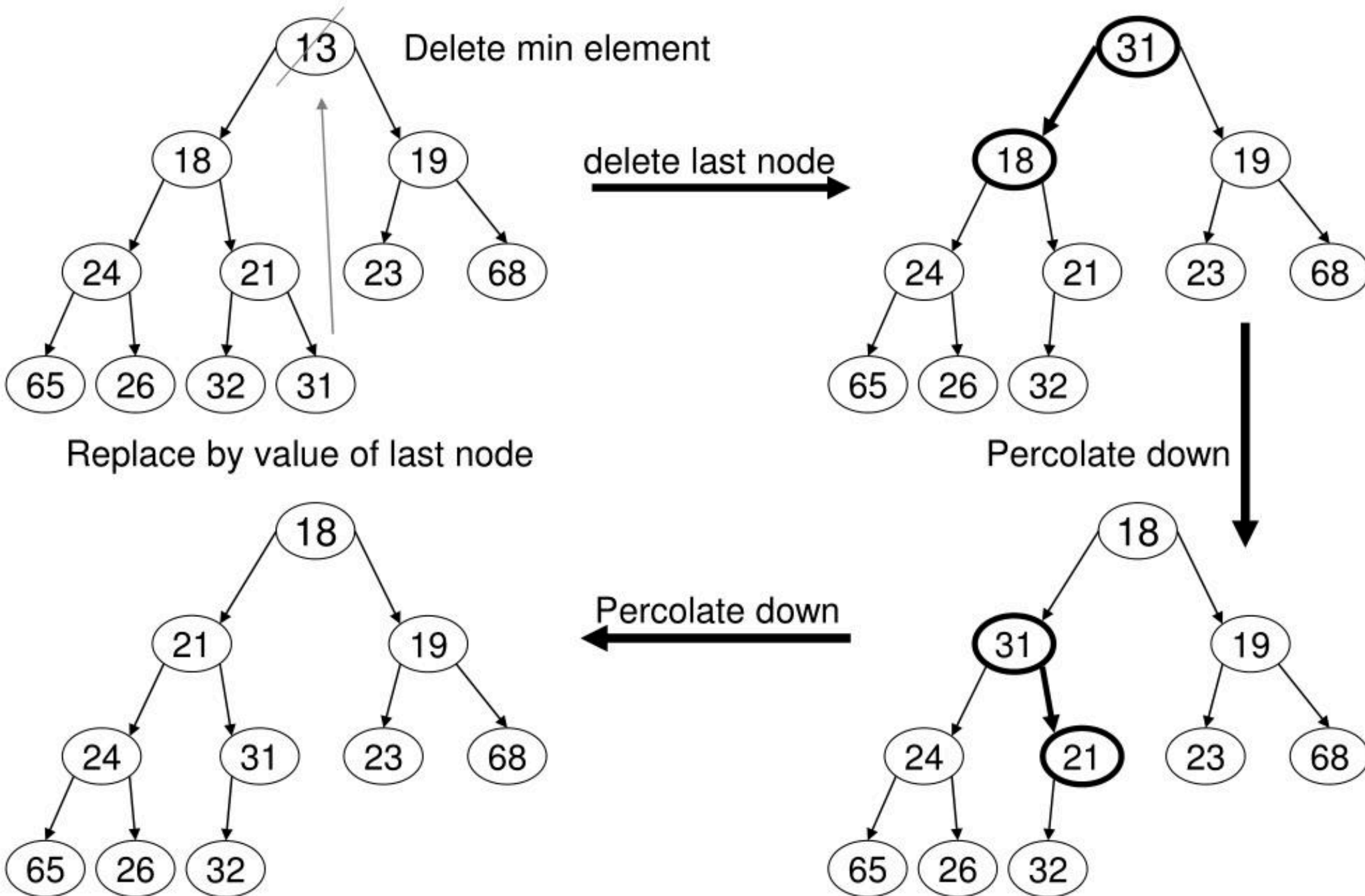
- 힙 순서 속성을 만족하는 완전 이진 트리
 - 완전 이진 트리: 최고 깊이를 제외한 모든 깊이의 노드가 완전히 채워져 있는 이진 트리이며, 마지막 레벨의 노드들은 왼쪽부터 차례대로 채워져 있어야 함.
 - 힙 순서 속성: 트리 내의 모든 노드가 부모 노드보다 커야 한다. (최소 힙(min heap)일 때)
 - 최대 힙은 루트 값이 가장 크고, 모든 자식 노드는 자신의 부모 노드보다 작은 값을 가짐.



이진탐색이 가능한가?

hips 삭제 연산

MinHeap Deletion Example



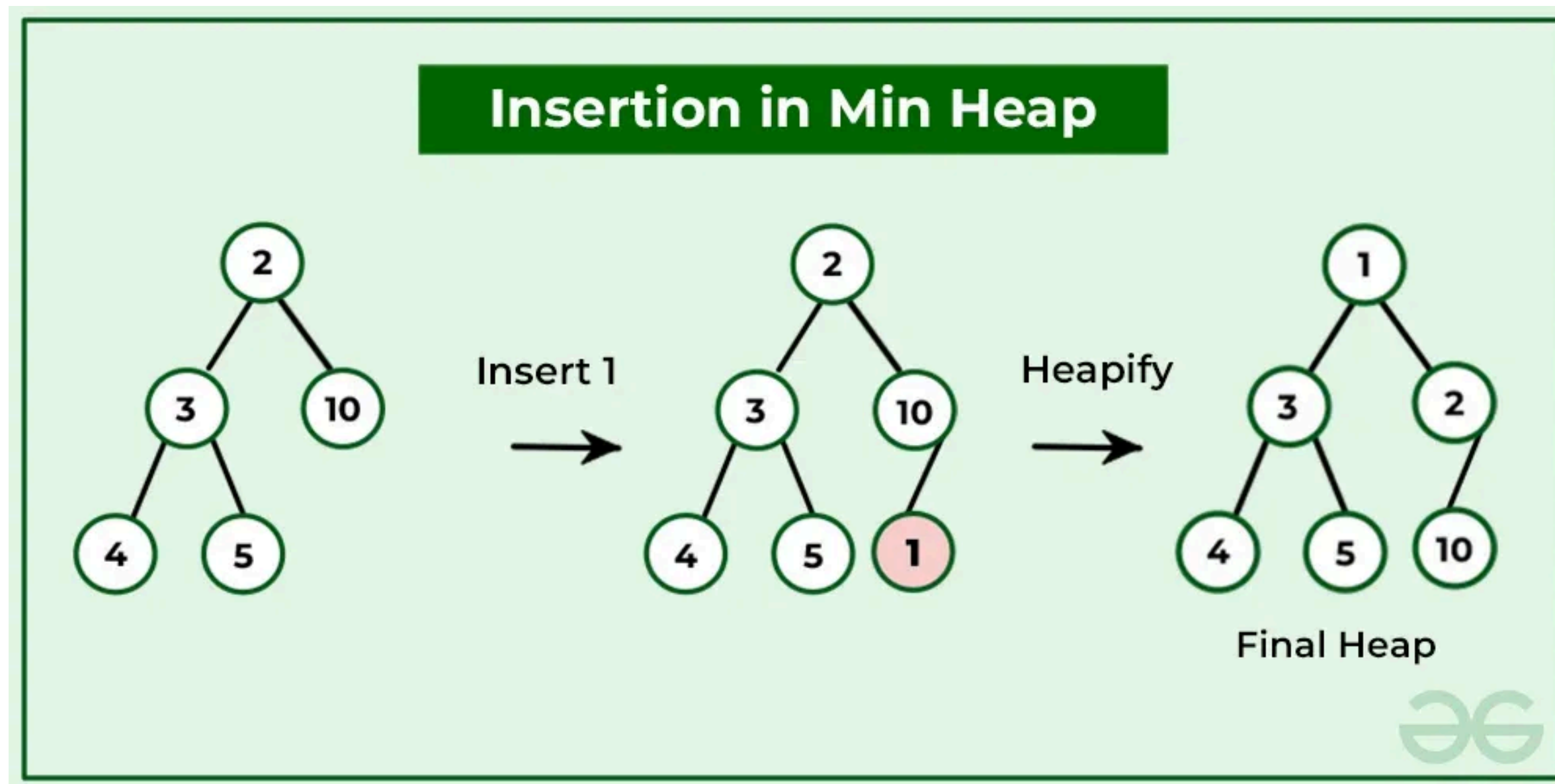
- 힙에서 루트 노드는 가장 우선순위가 높은(가장 작은 값) 노드다.
- 루트 노드를 지우고 → 마지막 노드를 올리고 → 내려가면서 마지막 노드의 적절한 위치를 찾기
- 루트 노드 삭제 후 작업
 - 1) 힙의 최고 값이 가장 우측에 있던 노드를 루트 노드로 옮긴다. (이 때, 힙 순서 속성이 파괴된다. 이를 복원해야 함.)
 - 2) 옮겨온 노드의 양쪽 자식을 비교하여 작은 쪽 자식과 위치를 교환한다.
 - 3) 2)를 반복하다가, 옮겨온 노드가 단말 노드가 되거나, 양쪽 자식보다 작은 값을 갖는 경우 삭제 연산 종료.

02

힙(Heap)의 연산

힙의 삽입 연산

- 마지막 노드로 추가하고 → 자기 위치 찾아서 보글보글 이동
- 삽입 연산의 작업
 - 1) 힙의 최고 깊이 가장 우측에 새 노드를 추가
 - 2) 삽입한 노드를 부모 노드와 비교한다. 삽입한 노드가 부모 노드보다 크면 연산 종료, 삽입한 노드가 부모 노드보다 작으면 3) 수행
 - 3) 삽입한 노드와 부모 노드의 자리를 맞교환하고, 2)를 반복



03

힙(Heap) 구현하기

- 우선순위 큐 ADT를 구현하는 방법이 힙
- 우선순위 큐: ADT
- 힙: 자료구조
- 힙을 구현하고
- 힙에 5,2,4,7,10,1,8,3,9 순서대로 추가한 후
- 다섯 개의 값을 꺼내 온 뒤,
- 힙의 값들을 순회하시오.(BFS)

01

응급실 환자 관리 프로그램

- 앞서 구현한 힙을 활용하여 응급실에서 환자 대기열을 관리하는 프로그램을 작성하시오.
- 새로운 환자가 대기열에 온다: enqueue
- 진료를 시작한다: dequeue
- 우선순위 점수 계산 방식
 - 중증도: 1~5단계, $\times 10$ 가산
 - 나이: 10세 이하 또는 65세 이상인 경우 +5 가산
 - 의식 수준: 무의식 / 혼미 / 정상, 각각 +10 / +5 / +0 가산



02-A

다익스트라 알고리즘 구현

- 여러 도시가 도로로 연결된 그래프가 주어지고, 각 도로에는 이동 비용(거리)이 주어진다. 시작 도시에서 다른 모든 도시까지의 최소 이동 비용을 구하는 프로그램을 작성하시오.
- 다익스트라 알고리즘은 매번 가장 짧은 거리의 노드를 꺼내야 한다.
- 힙을 활용하여 다익스트라 알고리즘을 구현하라.
- 단, 간선의 가중치 중 음수는 없다.
- 시작 도시를 입력 받고, 시작 도시에서 각 도시까지의 최단 거리를 출력하라.

• 입출력 예시

입력:

6 9	← 도시 수, 도로 수
1	← 시작 도시
1 2 2	← 도시A 도시B 비용 (단방향)
1 3 5	
1 4 1	
2 3 3	
2 4 2	
3 5 1	
4 3 3	
4 5 4	
5 6 2	

출력:

1번 도시까지:	0
2번 도시까지:	2
3번 도시까지:	5
4번 도시까지:	1
5번 도시까지:	6
6번 도시까지:	8

02-B

라면 공장

- 라면 공장에서는 하루에 밀가루를 1톤씩 사용합니다. 원래 밀가루를 공급받던 공장의 고장으로 앞으로 k 일 이후에야 밀가루를 공급받을 수 있기 때문에 해외 공장에서 밀가루를 수입해야 합니다.
- 해외 공장에서는 향후 밀가루를 공급할 수 있는 날짜와 수량을 알려주었고, 라면 공장에서는 운송비를 줄이기 위해 최소한의 횟수로 밀가루를 공급받고 싶습니다.
- **현재 공장에 남아있는 밀가루 수량 $stock$, 밀가루 공급 일정 ($dates$)과 해당 시점에 공급 가능한 밀가루 수량 ($supplies$), 원래 공장으로부터 공급받을 수 있는 시점 k 가 주어질 때, 밀가루가 떨어지지 않고 공장을 운영하기 위해서 최소한 몇 번 해외 공장으로부터 밀가루를 공급받아야 하는지를 `return` 하도록 `solution` 함수를 완성하세요.**
- $dates[i]$ 에는 i 번째 공급 가능일이 들어있으며, $supplies[i]$ 에는 $dates[i]$ 날짜에 공급 가능한 밀가루 수량이 들어 있습니다.

• 제한사항

- $stock$ 에 있는 밀가루는 오늘(0일 이후)부터 사용됩니다.
- $stock$ 과 k 는 2 이상 100,000 이하입니다.
- $dates$ 의 각 원소는 1 이상 k 이하입니다.
- $supplies$ 의 각 원소는 1 이상 1,000 이하입니다.
- $dates$ 와 $supplies$ 의 길이는 1 이상 20,000 이하입니다.
- k 일 째에는 밀가루가 충분히 공급되기 때문에 $k-1$ 일에 사용할 수량까지만 확보하면 됩니다.
- $dates$ 에 들어있는 날짜는 오름차순 정렬되어 있습니다.
- $dates$ 에 들어있는 날짜에 공급되는 밀가루는 작업 시작 전 새벽에 공급되는 것을 기준으로 합니다. 예를 들어 9일째에 밀가루가 바닥나더라도, 10일째에 공급받으면 10일째에는 공장을 운영할 수 있습니다.
- 밀가루가 바닥나는 경우는 주어지지 않습니다.

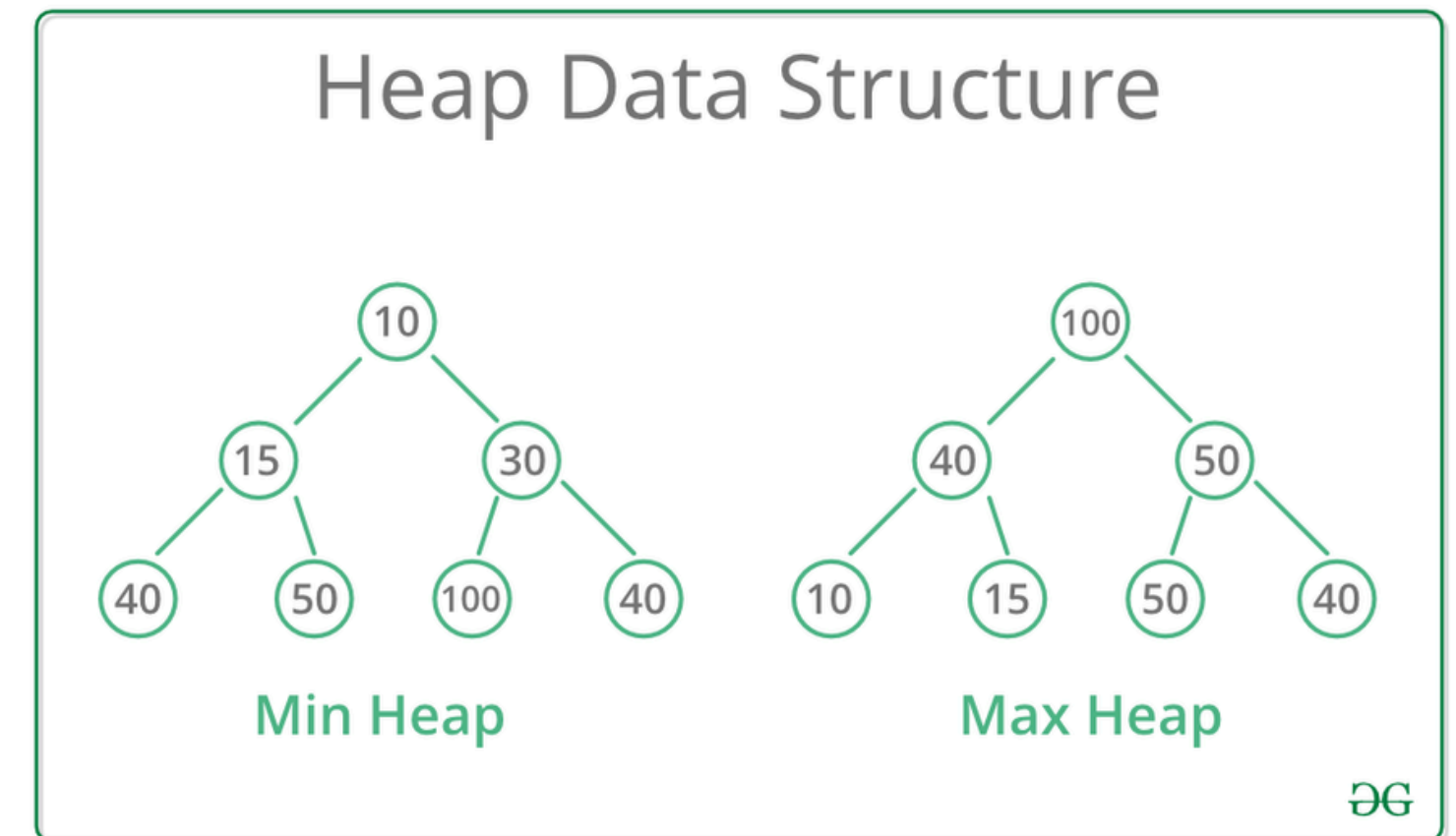
• 입출력 예시

- $stock=4$, $dates=[4,10,15]$, $supplies=[20,5,10]$, $k=30$ 일 때
- 정답: 2

01

탐구 내용 돌아보기

- 응급실, 다익스트라, 라면 공장에서 각각 힙을 어떤 식으로 사용했는지 돌아 봅시다.
 - 응급실: 환자들 우선순위를 관리하고, 우선순위가 가장 높은 환자를 꺼낸다.
 - 다익스트라: 매번 가장 가까운(비용이 적은) 노드를 꺼낸다.
 - 라면 공장: 재고가 바닥나기 전, 가장 많은 양을 꺼낸다.
- 반복적으로 최솟값 또는 최댓값을 꺼내는 상황
- dequeue 작업의 빅 오는?
- enqueue 작업의 빅 오는?
- 만약 힙이 아니라면???



02

힙의 강점

- 루트가 항상 최솟값/최댓값임을 보장해서, 값을 꺼내 쓸 때 효율적이다.
- 특히 데이터가 많을수록, 여러 번 반복할수록 극적인 효과
- 최솟값/최댓값을 빠르게 꺼내는 것 하나에 특화된 구조
- 데이터가 실시간으로 추가되는 상황에서도 강하다.
- 어떤 문제에 힙을 써야 하는가?
 - 최소 비용으로~ / 최소 횟수로~
 - 매번 가장 ~한 것을 선택한다.
 - 우선순위에 따라 처리한다.
 - 실시간 데이터 추가 + 최솟값/최댓값

03

힙의 다른 면

- 루트만! 최소값(또는 최대값)만을 보장한다. 루트만!
- 전체가 정렬되어 있는 것이 아니다.
- 힙(만)으로 해결이 어려운 경우
 - 특정 값을 검색해야 할 때
 - 순서가 중요할 때
 - 중간값이 필요할 때
 - 전체를 정렬된 상태로 유지해야 할 때

04

모뎀 활동 정리 - 힙(Heap) 구현하기

```
1 heap = []
2
3 def push(val):
4     heap.append(val)
5     i = len(heap) - 1
6     while i > 0:
7         parent = (i - 1) // 2
8         if heap[i] < heap[parent]:
9             heap[i], heap[parent] = heap[parent], heap[i]
10            i = parent
11        else:
12            break
13
14 def pop():
15     if len(heap) == 1:
16         return heap.pop()
17     val = heap[0]
18     heap[0] = heap.pop()
19     i = 0
20     while True:
21         left, right, smallest = 2*i+1, 2*i+2, i
22         if left < len(heap) and heap[left] < heap[smallest]: smallest = left
23         if right < len(heap) and heap[right] < heap[smallest]: smallest = right
24         if smallest == i: break
25         heap[i], heap[smallest] = heap[smallest], heap[i]
26         i = smallest
27     return val
```

04

모뎀 활동 정리 - 다익스트라

- 1. 시작 노드의 거리를 0, 나머지는 INF로 초기화
- 2. (거리, 노드)를 힙에 삽입
- 3. 힙이 빌 때까지:
 - a. 힙에서 (현재 거리, 현재 노드) 꺼내기
 - b. 현재 거리 > 이미 알고 있는 최단 거리 → 스킵
 - c. 현재 노드의 인접 노드 탐색 - 새 거리 = 현재 거리 + 간선 가중치 - 새 거리 < 기존 최단 거리 → 갱신 후 힙에 삽입

04

모뎀 활동 정리 - 라면 공장

- 1. 현재 재고로 버틸 수 있는 날(day)을 추적
- 2. day가 k보다 작은 동안 반복:
 - a. day까지 공급 가능한 날짜들을 모두 힙에 추가
 - b. 힙에서 가장 많은 수량을 꺼내 day에 더함
 - c. 수입 횟수 +1
- 3. day $\geq$ k가 되면 종료

Q&A
